CS 5330 HW5

1. Give an intuitive definition of the meaning of eigenvectors in the context of image analysis (think about the faces example).

   Eigenvectors of the covariance matrix tell us the principal directions of variation in the data and their relative importance. In the context of image analysis, an eigenvector might represent the differences in one of the several different facial features (eyebrow, mouth, eye, etc.).

2. You have the set of eigenvalues 0.3, 0.25, 0.24, 0.12, 0.06, 0.02, 0.01

   a. What is the dimensionality of original data (the number of significant dimensions)?

      7 (each eigenvalue has one eigenvector)

   b. How many dimensions of the data would you need to keep to represent 75% of the variation?

      The first 3 (0.3+0.25+0.24 = 0.79)

   c. How many dimensions of the data would you need to keep to represent 90% of the variation?

      The first 4 (0.3+0.25+0.24+0.12 = 0.91)

   d. If you wanted to see best visualization of the data in 2-D, how would you do it?

      Use the top two eigenvectors (1st and 2nd directions of most variation) as the two axies and project all datapoints to that plane.

3. Consider a training set of images of doors. Each image is 128×128 = 2^14 pixels. You decide to build an image eigenspace of reduced dimension using your data set. You find that 32 eigenvectors represents over 90% of the variation in the data set and decide to use those eigenvectors as your embedding space.

   a. What is the process for projecting a new image of a door into the door eigenspace?

First, compute the mean image by averaging all pixels across all images in the training set.

Then that image is subtracted by the image mean.

Then the resulting image is projected to the eigenspace by taking the dot product with every eigenvector (this gives 32 scalars which become the new “coordinates” in eigenspace)

b. How many dimensions will the embedding of the new image have? What is the compression factor?

   The new image will have 32 dimensions. The compression factor is **16384 / 32 = 512**

c. How big is each eigenvector?

   each eigenvector has a dimension the size of the image, so 2^14 = 16384

d. What is the process for re-creating the original image from the embedding?

   To recreate the original image, the eigenspace projection (scalars) are multiplied by each of their corresponding eigenvectors. Then the 32 vectors are added together (resulting in one 2^14 dimension vector) and the mean image is added back (since the mean image was subtracted during projection).

e. If you project a picture of a frog into the door eigenspace and then try recreating the original image, what will the re-creation look like?

   The frog will look somewhat like a door, or at least the closest thing to a frog that the eigenspace can represent. The eigenspace only knows how to represent the variation that exists within door images in the training set

4. Why is connectedness important in binary image processing? Given an example as to how it might affect something like segmentation, connected components analysis, or morphological filtering.

   There are different types of connectedness, (4-way vs 8-way) which each influence the output of things like segmentation, or morphological filtering. 4-way connectedness will fail to connect segments of an image that share corners, but not edges. Likewise, during morphological filtering operations like

dilution and shrinking, the connectedness type will influence if neighboring corners are included in the operation. Other types of connectedness can also treat non-adjacent pixels as connected too, such as the 5×5 connectedness type demonstrated in lecture.

5. What is the purpose of the morphological operators (A) growing/dilation, and (B) shrinking/erosion?

   The two operations are useful if trying to extract features from an image based on their shape. Growing/dilation will “expand” the boundary of a region into connected pixels, while shrinking/erosion will do the opposite. Shrinking is an effective way to remove noisy “blobs” from an image, while growing is an effective way to merge two shapes that are very close but not quite touching

6. What is the purpose of the grassfire transform algorithm? Describe the inputs and outputs.

   The grassfire transform algorithm is a useful way to get the distance of a pixel to the nearest border of the region it is in (this in itself is useful knowledge because it allows for shrinking/growing to be done quickly in a single pass. The inputs are an image, and the outputs are a new “distance” image which holds the information about the distance of each pixel to the closest boundary - or the distance to the background.

7. When is it more efficient to grow or shrink using the grassfire transform compared to using a standard morphological filter?

   It is more efficient to do one grow or shrink operation if the distance to the background of each pixel is known. This means that, for example, 5 pixels can be shrunk in a single pass by simply deleting the pixels with distance values less than 5, rather than having to do 5 separate passes as in morphological filtering. The grassfire transform itself is very computationally efficient because it only looks at two pixels for each one pixel in the image (above and behind), and only does two passes over the image.

8. How is building an embedding space using PCA different than building one using a deep network like ResNet18? Your answer can discuss process or quality.

   PCA uses the X most “important” (determined by some target percentage, i.e. 90%) eigenvectors (which hold key information about the direction of most

variance in the training set) to build an image embedding, DNN uses layers of filtering, some morphological, some not, applied over the image training set. These filters are learned and not derived from purely statistical observations as with PCA. PCA embedding is often faster and more “intuitive” while DNN is slower but of higher quality